

ACTIVITY - I

I. What is API testing?

API (Application Programming Interface) testing is a type of software testing that involves verifying that the communication between two systems happens correctly. Since APIs don't have a user interface (UI), you send a request to a specific endpoint and validate the response.

Real-Time Example: Booking a Flight

Imagine you are using a travel site like **Expedia** to find a flight from New York to London.

1. **The Request:** When you click "Search," Expedia sends an API request to various airlines (Delta, United, British Airways).
 - **Endpoint:** <https://api.airline.com/v1/flights>
 - **Method:** **GET**
 - **Parameters:** **origin=JFK, destination=LHR, date=2026-06-15**
2. **The Response:** The airline's system checks its database and sends back data, usually in **JSON** format.
 - **Status Code:** **200 OK** (Everything went fine).
 - **Data:** A list of flight times, seat availability, and prices.

What are we actually testing?

In this flight booking scenario, an API tester checks for:

- **Accuracy:** Does the price in the API match what is shown on the screen?
- **Security:** If I try to book a flight without an "API Key" (authentication), does the system block me?
- **Error Handling:** If I enter a date in the past, does the API return a clear error message like **400 Bad Request**?
- **Performance:** Does the API respond within 2 seconds, or does it hang?

Core Components of an API Test

Component	Description	Example
Endpoint	The URL where the service lives.	/api/login
Method	The action you want to perform.	GET, POST, PUT, DELETE
Headers	Metadata (Authorization, Content-Type).	Bearer: xyz123token

Body	The data sent to the server (for POST/PUT).	{"username": "johndoe"}
Status Codes	The server's answer.	200 (Success), 404 (Not Found), 500 (Server Error)

Why is it so important?

1. **Earlier Testing:** You don't have to wait for the UI/Website to be finished. You can test the logic as soon as the code is written.
2. **Speed:** Running 100 UI tests might take an hour; running 100 API tests takes seconds.
3. **Reliability:** If the API works, the "brain" of the application is healthy. UI changes (like moving a button) won't break your API tests.

II. What you can do in API?

an API (Application Programming Interface) allows you to **borrow** features or data from another software so you don't have to build them yourself.

1. Connect Different Apps (Integration)

The most common use for an API is making two separate programs talk to each other.

- **The "Login with Google/Facebook" Button:** Instead of building a complex "Sign Up" and "Forgot Password" system, you use the Google API to let users log in using their existing Google account.
- **E-commerce & Payments:** When you buy something on a small website, they don't handle your credit card directly. They use the **Stripe** or **PayPal API** to process the payment safely.

2. Automate Boring Tasks

APIs allow you to set up "Triggers" so that when something happens in one app, another app automatically reacts.

- **Customer Support:** When a customer sends an email, an API can automatically create a ticket in **Zendesk** and notify the team on **Slack**.
- **Social Media:** You can write a script using the **X (Twitter) API** to automatically post a "Good Morning" message every day at 8:00 AM without you ever opening the app.

3. Access Real-Time Data

You can pull live information from giant databases to display on your own app.

- **Weather:** A local news app doesn't have its own satellites. It uses the **OpenWeather API** to get live forecasts.

- **Stock Market/Crypto:** Apps like Robinhood use APIs to get the literal second-by-second price of Bitcoin or Apple stock.

4. Add Advanced "Brainpower" (AI)

You don't need to be a math genius to add Artificial Intelligence to your project.

- **ChatGPT in your app:** You can use the **OpenAI API** to add a chatbot to your website that can answer customer questions or summarize long documents.
- **Image Recognition:** You can use an API to "read" photos—for example, automatically identifying if a photo uploaded to your site contains a face or a specific object.

III. What is the status code in the current market ?

Status codes are divided into 5 "Series" based on their first digit:

1. The 2xx Series: "Everything is Awesome" (Success)

These tell you the request was received and processed correctly.

- **200 OK:** The most common code. Everything worked, and the data is in the response.
- **201 Created:** Success! A new resource (like a user account or a post) was successfully built.
- **204 No Content:** The request worked, but there's nothing to send back (common when you Delete something).

2. The 3xx Series: "Go Over There" (Redirection)

The server is telling the client that the resource has moved.

- **301 Moved Permanently:** The URL you called is gone forever; use the new one provided.
- **304 Not Modified:** Tells the browser: "The data hasn't changed since you last checked, just use your saved (cached) version." (Great for saving mobile data).

3. The 4xx Series: "You Messed Up" (Client Errors)

These occur when the request sent by the user/app is wrong.

- **400 Bad Request:** The server can't understand the request (e.g., you forgot a required field like an email address).
- **401 Unauthorized:** "I don't know who you are." You forgot to log in or provide an API key.
- **403 Forbidden:** "I know who you are, but you aren't allowed here." (e.g., a regular user trying to access an Admin page).
- **404 Not Found:** The most famous code. The URL or specific resource doesn't exist.
- **429 Too Many Requests:** You are calling the API too fast. This is the standard "Rate Limit" error in 2026.

4. The 5xx Series: "I Messed Up" (Server Errors)

The request was fine, but the server crashed or failed.

- **500 Internal Server Error:** The "catch-all" for when something breaks in the server's code.
- **502 Bad Gateway:** One server got an invalid response from another (common in cloud environments like AWS or Azure).
- **503 Service Unavailable:** The server is overloaded or down for maintenance.
- **504 Gateway Timeout:** The server took too long to respond, and the connection died.

IV. what is http and https?

1. HTTP (HyperText Transfer Protocol)

HTTP is the foundation of the World Wide Web. It is a system for transmitting data (text, images, video) across the internet.

- **How it works:** It uses a "Request-Response" model. You click a link (Request), and the server sends you the webpage (Response).
- **The Flaw:** HTTP is **unsecured**. Data is sent in "Plain Text."
- **Real-world Risk:** If you enter your password on an [http://](#) site, a hacker sitting on the same Wi-Fi network could technically "sniff" the data and read your password exactly as you typed it.

2. HTTPS (HyperText Transfer Protocol Secure)

HTTPS is the secure version of HTTP. It uses a security protocol called **TLS (Transport Layer Security)**—formerly known as SSL—to wrap your data in an encrypted "envelope."

- **Encryption:** It scrambles the data. Even if a hacker intercepts it, they only see a random string of characters like [83hf!92#kd](#).
- **Authentication:** It proves that the website is who it says it is. This is verified by an **SSL Certificate**.
- **Data Integrity:** It ensures that the data wasn't tampered with while traveling from the server to your screen.

Feature	HTTP	HTTPS
Full Name	HyperText Transfer Protocol	HyperText Transfer Protocol Secure

Security	None (Plain Text)	High (Encrypted)
Port Used	Port 80	Port 443
SEO Impact	Low (Google penalizes it)	High (Google ranks it better)
Browser Icon	"Not Secure" warning	A Padlock icon 

In 2026, HTTPS is no longer "optional."

1. **Browser Warnings:** Chrome, Safari, and Firefox now show giant red warnings if you try to visit an HTTP site, which scares away customers.
2. **Modern APIs:** Almost every modern API (like the ones we discussed earlier) **requires** HTTPS. You cannot securely send a payment or a private message over standard HTTP.
3. **Speed:** HTTPS actually allows the use of a newer, faster protocol called **HTTP/2** (and HTTP/3), which makes websites load significantly quicker than old HTTP.

V. Types of Functional API testing:

1. Unit Testing

This is the most basic level. It tests a **single endpoint** in isolation to ensure it performs its specific task.

- **Real-time Example:** Testing only the **GET /user/profile** endpoint to see if it returns the correct name and email for User ID 123.
- **Goal:** To verify that the individual "unit" of code works before connecting it to anything else.

2. Integration Testing

This is where things get interesting. Most software features require multiple APIs to talk to each other. Integration testing checks the **communication between these units**.

- **Real-time Example:** After a user signs up (**POST /register**), can they immediately log in (**POST /login**)?
- **Goal:** To ensure data flows correctly from one API to the next without getting corrupted.

3. End-to-End (E2E) Testing

This tests a complete **user journey** from start to finish. It mimics a real human using the app.

- **Real-time Example (Amazon/E-commerce):**
 1. Search for a product (**GET /search**).
 2. Add to cart (**POST /cart**).
 3. Enter shipping info (**PUT /shipping**).
 4. Complete payment (**POST /checkout**).
- **Goal:** To make sure the entire "business logic" is solid across the whole system.

4. Boundary Value Analysis (Negative Testing)

This is "testing the edges." You check how the API handles data that is just barely valid or clearly invalid.

- **Real-time Example:** If a "Password" field requires 8 to 16 characters, you test with:
 - 7 characters (Should fail/Error 400).
 - 8 characters (Should pass/Success 200).
 - 17 characters (Should fail/Error 400).
- **Goal:** To ensure the API has strict rules and doesn't crash when it gets unexpected input.

5. Error Condition Testing

In functional testing, we don't just test for success; we test for **graceful failure**.

- **Real-time Example:** Trying to delete a post that has already been deleted.
- **Expected Result:** The API shouldn't crash; it should return a clear **404 Not Found** or **410 Gone** message.